

Levantamento de Requisitos

Template para preencher antes de começar o projeto final.

Projeto:
Equipe:
Data:

Preencha cada bloco com o tema real do grupo. Os exemplos existem apenas para guiar; substitua tudo pela versão do seu projeto.

1. Público-alvo

Para quem é esse sistema? Quem vai usar no dia a dia?

Exemplo: estudantes e profissionais que querem organizar suas tarefas pessoais em um só lugar.

Escreva aqui.

2. Problema que o projeto resolve

Qual dor ou dificuldade desse público o sistema vem resolver?

Exemplo: hoje as tarefas ficam espalhadas em papéis e apps diferentes, o que gera esquecimento e perda de prazos.

Escreva aqui.

3. Solução proposta

Em poucas linhas, o que o sistema vai fazer para resolver esse problema?

Exemplo: uma API de lista de tarefas onde cada usuário cria, acompanha e conclui suas tarefas em um só lugar, com acesso seguro por conta individual.

Escreva aqui.

4. Objetivos do projeto

O que o projeto precisa alcançar? Use metas claras sempre que possível.

Exemplo: centralizar as tarefas do usuário; garantir que cada usuário veja apenas as próprias tarefas; permitir cadastrar uma tarefa em menos de 30 segundos.

Escreva aqui.

5. Escopo

Deixe claro o que entra e o que não entra nesta entrega.

O sistema fará

Exemplo: cadastro e login, criar/listar/editar/excluir tarefas e marcar tarefa como concluída.

Escreva aqui.

O sistema não fará

Exemplo: compartilhar tarefas entre usuários, enviar notificações por e-mail ou integrar calendário externo.

Escreva aqui.

6. Restrições e premissas técnicas

Quais limitações o projeto precisa respeitar? Linguagem, banco, padrões obrigatórios, prazo, autenticação, deploy e frontend.

Exemplo: backend em Node.js com Express e MongoDB; autenticação obrigatória via JWT; backend e frontend publicados no Render; frontend com FlutterFlow ou IA; entrega até a data definida em aula.

Escreva aqui.

7. Atores / tipos de usuário

Quem interage com o sistema e o que cada um pode fazer?

Ator	O que ele pode fazer
Usuário autenticado	Gerencia apenas os próprios registros.

8. Telas / interfaces ou endpoints

Quais telas o sistema terá e o que o usuário faz em cada uma? Se o grupo começar pelo backend, descreva os principais endpoints planejados.

Tela ou endpoint	O que o usuário faz	Quem acessa
Tela de Login	Entrar no sistema com e-mail e senha.	Visitante

9. Entidades / dados principais

Quais dados o sistema vai guardar? Liste as principais entidades e seus campos mais importantes. O projeto final deve ter pelo menos 4 entidades do domínio além de **Usuario**.

Entidade	Principais informações	Relaciona-se com
Tarefa	título, descrição, status, data de criação	pertence a um Usuário

9.1 Relacionamentos entre entidades

Explique como as entidades se conectam.

Exemplo: um Pedido pertence a um Cliente; um Pedido possui vários ItensPedido; cada ItemPedido referencia um Produto.

Relacionamento	Como funciona
----------------	---------------

10. Requisitos Funcionais

Requisitos funcionais descrevem o que o sistema deve fazer. Use frases específicas e testáveis.

ID	Descrição	Prioridade
RF-001	O sistema deve permitir o cadastro com e-mail e senha, recusando e-mails já existentes.	Essencial
RF-002	O sistema deve permitir login e retornar um token JWT válido.	Essencial

11. Requisitos Não Funcionais

Requisitos não funcionais descrevem como o sistema deve se comportar: segurança, organização, desempenho, usabilidade e confiabilidade.

ID	Descrição	Prioridade
RNF-001	As senhas devem ser armazenadas com hash, nunca em texto puro.	Essencial

ID	Descrição	Prioridade
RNF-002	O backend deve usar camadas separadas para model, repository, service, controller e routes.	Essencial

12. Regras de Negócio

Regras de negócio são políticas e restrições específicas do domínio do projeto.

ID	Descrição	RF relacionado
RN-001	Um usuário só pode ver e modificar os próprios registros.	RF-003

13. Guia de priorização

Use esta escala:

Prioridade	Significado
Essencial	Sem isso o sistema não funciona. Tem que ser feito.
Importante	O sistema funciona sem isso, mas fica incompleto. Deve ser feito se houver tempo.
Desejável	Não compromete o funcionamento. Pode ficar para uma versão futura.

14. Checklist antes de programar

- ☐ O problema está claro.
- ☐ O público-alvo está definido.
- ☐ O escopo diz o que entra e o que fica fora.
- ☐ As entidades do domínio foram escolhidas.
- ☐ Existem pelo menos 4 entidades do domínio além de Usuario.
- ☐ Os relacionamentos entre entidades foram definidos.
- ☐ Pelo menos 2 entidades terão CRUD completo.
- ☐ Existe pelo menos 1 regra de negócio envolvendo mais de uma entidade.
- ☐ Backend e frontend serão publicados no Render.
- ☐ As telas ou endpoints principais estão planejados.
- ☐ Os requisitos funcionais estão escritos de forma testável.
- ☐ As regras de negócio foram discutidas.
- ☐ O grupo sabe qual é o mínimo para entregar uma versão funcional.